Symulator obsługi transakcji PoS

Vladyslav Rosol i Olesia Voronina

Grupa 22 INF-SP/B, Podstawy informatyki

C++17. FIFO i FEE w czasie dyskretnym.

1. Cel i zasady modelu

Program porównuje FIFO i FEE w systemie, który ma więcej zadań do wykonania niż dostępnych stanowisk. Zgłoszeniami są transakcje, a wykonawcami walidatorzy. Sprawdzamy oczekiwanie, dotrzymanie terminów i wykorzystanie wykonawców przy różnym tempie napływu transakcji.

Czas ma kolejne wartości całkowite: 0, 1, 2 i następne. Jeden krok jest umowną jednostką. Program oblicza przebieg i nie czeka rzeczywistej sekundy między krokami.

Transakcja ma ID, nadejście, czas wykonania, opłatę fee i deadline. Walidator ma ID, dodatni stake i prywatny stan pracy. Może wykonywać jedną transakcję naraz. Wszyscy walidatorzy pracują z jednakową szybkością. Rozpoczętej pracy nie przerywamy, a spóźnione zadania kończymy i oznaczamy w wyniku.

Wybór transakcji i wykonawcy to osobne decyzje. FIFO pobiera wcześniejsze nadejście, a przy remisie mniejsze ID. FEE pobiera większe fee, następnie wcześniejsze nadejście i mniejsze ID. Spośród wolnych wykonawców losujemy walidatora proporcjonalnie do jego stake. Szansa wynosi stake walidatora podzielony przez sumę stake wolnych walidatorów.

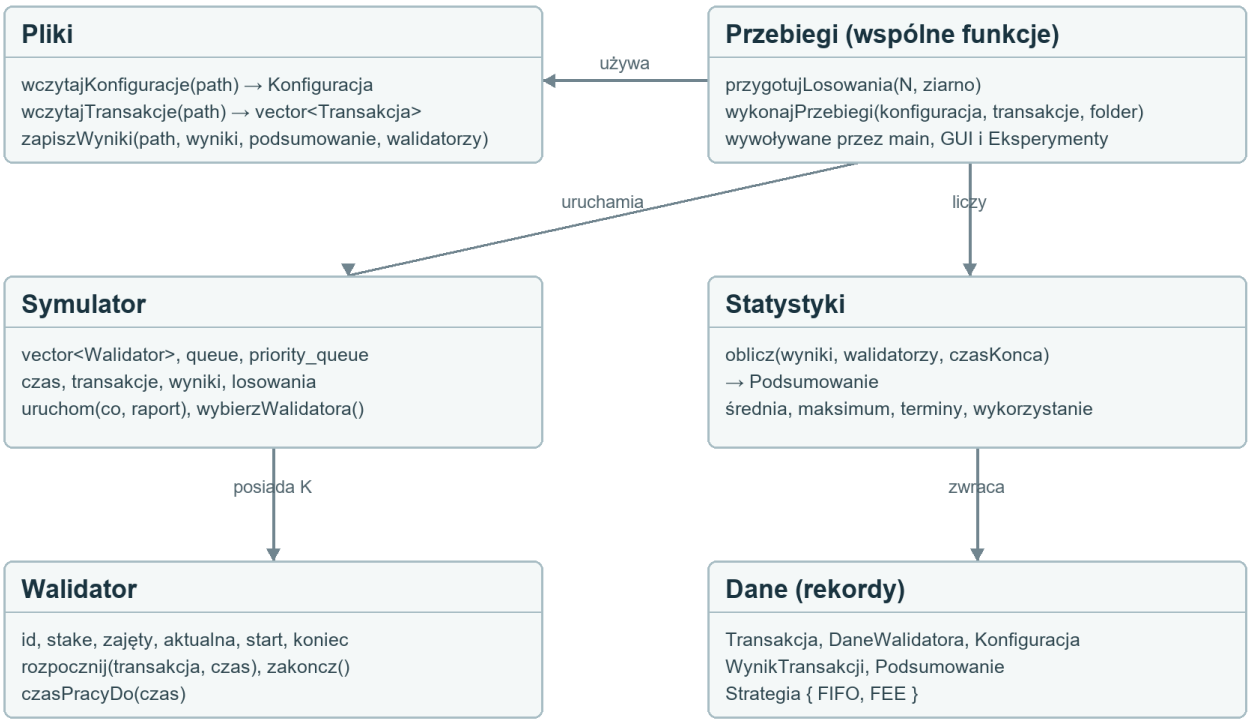
Kontekst PoS ograniczamy do takiego losowania wykonawcy. Model nie tworzy bloków, nie weryfikuje podpisów, nie obsługuje sieci i nie odtwarza konsensusu blockchaina. Stake pozostaje stały w całym przebiegu.

Przebieg jednego kroku

Dodajemy transakcje, które właśnie nadeszły. Dopóki są zadania i wolni wykonawcy, losujemy wykonawcę i pobieramy zadanie z wybranej kolejki. Jeśli została praca lub przyszłe nadejścia, zwiększamy czas o jeden, zapisujemy zakończenia i zwalniamy wykonawców. Potem wracamy do nadejść.

Transakcja rozpoczęta w 0 i trwająca 3 kroki kończy się w 3. W tym samym kroku wykonawca może rozpocząć kolejne zadanie. Pełny diagram działania znajduje się w diagramy.pdf.

2. Klasy i struktury danych



Cztery klasy obejmują stan wykonawcy, przebieg, pliki i pomiary. Typy wspólne są prostymi rekordami. Nie potrzebujemy dziedziczenia ani ręcznego przydzielania pamięci.

Element	Odpowiedzialność
Walidator	Rozpoczęcie i zakończenie jednego zadania; czas pracy do chwili pomiaru.
Symulator	Czas, kolejki, losowania i zbieranie zakończeń. Posiada K obiektów Walidator.
Pliki	Odczyt, sprawdzenie pól, sortowanie wejścia i zapis wyników.
Statystyki	Końcowa średnia, maksimum, terminowość i wykorzystanie każdego wykonawcy.
Przebiegi	Funkcje wspólne: lista losowań oraz oddzielne uruchomienie FIFO i FEE.
Dane.hpp	Rekordy transakcji, walidatora, konfiguracji, wyniku i podsumowania.

vector przechowuje dane, wykonawców, losowania i wyniki. queue obsługuje FIFO. priority_queue używa funktora PorownajFee. Przed symulacją sortujemy transakcje po nadejściu i ID, a wykonawców po ID.

main.cpp obsługuje konsolę, Gui.cpp okno Windows, a Eksperymenty.cpp dane badawcze. Każdy korzysta z wykonajPrzebiegi. WidokWynikow.hpp sortuje indeksy tabeli bez zmiany oryginalnego wyniku. Sprawdzony diagram klas z FigJam jest w diagramy.pdf.

3. Dane i sposób przeprowadzenia badań

W każdym scenariuszu użyliśmy 6000 transakcji i 300 walidatorów. Dane są syntetyczne. Generator w C++ używa mt19937 z ziarnem 20261102. Dla danej transakcji zachowujemy ten sam czas wykonania i fee pomiędzy scenariuszami. Zmieniamy nadejście i przesuwamy deadline, zachowując ten sam zapas czasu.

Parametr	Wartość
ID	Transakcje 1 do 6000, walidatorzy 1 do 300.
Stake	10 + (ID × 37) mod 991, stały przez cały przebieg.
Czas wykonania	8 + generator() mod 25, czyli 8 do 32 kroków. Wartości powtarzają się.
Fee	1 + generator() mod 100, czyli 1 do 100.
Deadline	nadejście + czas wykonania + 10 + floor((100-fee)/2).

floor oznacza część całkowitą wyniku. Większe fee ma mniejszy zapas deadline: od 10 do 59 kroków. Jest to założenie tych danych, a nie cecha wszystkich systemów PoS.

Scenariusz	Nadejścia	Warunki pracy
Małe obciążenie	6 na krok, kroki 0 do 999.	Zapas zasobów; w danych nie ma oczekiwania.
Przeciążenie	100 na krok, kroki 0 do 59.	Kolejka rośnie, bo napływ przekracza wydajność.
Nagły napływ	Wszystkie 6000 w kroku 0.	Jednorazowa długa kolejka.

Przy czasie wykonania około 20 kroków 300 wykonawców obsługuje w przybliżeniu 15 zadań na krok. Sześć nowych zadań mieści się poniżej tej wydajności, a sto ją przekracza. Domyślny zgloszenia.txt jest kopią przeciazenie.txt.

Dla każdego scenariusza wykonaliśmy FIFO i FEE z ziarnami 11, 22 i 33. To 18 przebiegów. Jeden start programu liczy oba warianty, więc wystarcza dziewięć uruchomień. Każda strategia ma nowy obiekt Symulator i zaczyna od pierwszej pozycji wspólnej listy N losowań.

Wyniki zachowaliśmy w eksperymenty/scenariusz/ziarno. porownanie.csv zawiera miary całych przebiegów, wykorzystanie.csv dane każdego walidatora, a grupy_fee.csv podział na niskie i wysokie opłaty.

4. Miary

Oczekiwanie = start - nadejście. Opóźnienie = max(0, koniec - deadline). Zadanie kończące się dokładnie w deadline jest terminowe. Dla pustego wejścia wszystkie miary wynoszą zero.

Średnią i maksimum oczekiwania oraz liczbę terminowych i spóźnionych liczymy dla zakończonych transakcji. Wykorzystanie walidatora to 100% × jego czas pracy / czas od 0 do chwili pomiaru. Średnie wykorzystanie obejmuje wszystkich wykonawców, także tych bez zadań. Raporty co 100 są narastające, a wykorzystanie w trakcie obejmuje już wykonaną część zadań w toku. Ostatni raport obejmuje również niepełną setkę.

5. Wyniki wszystkich 18 przebiegów

Czasy są w krokach symulacji. W ostatniej kolumnie podajemy średnie wykorzystanie wszystkich 300 walidatorów. Dokładniejsze wartości są w CSV.

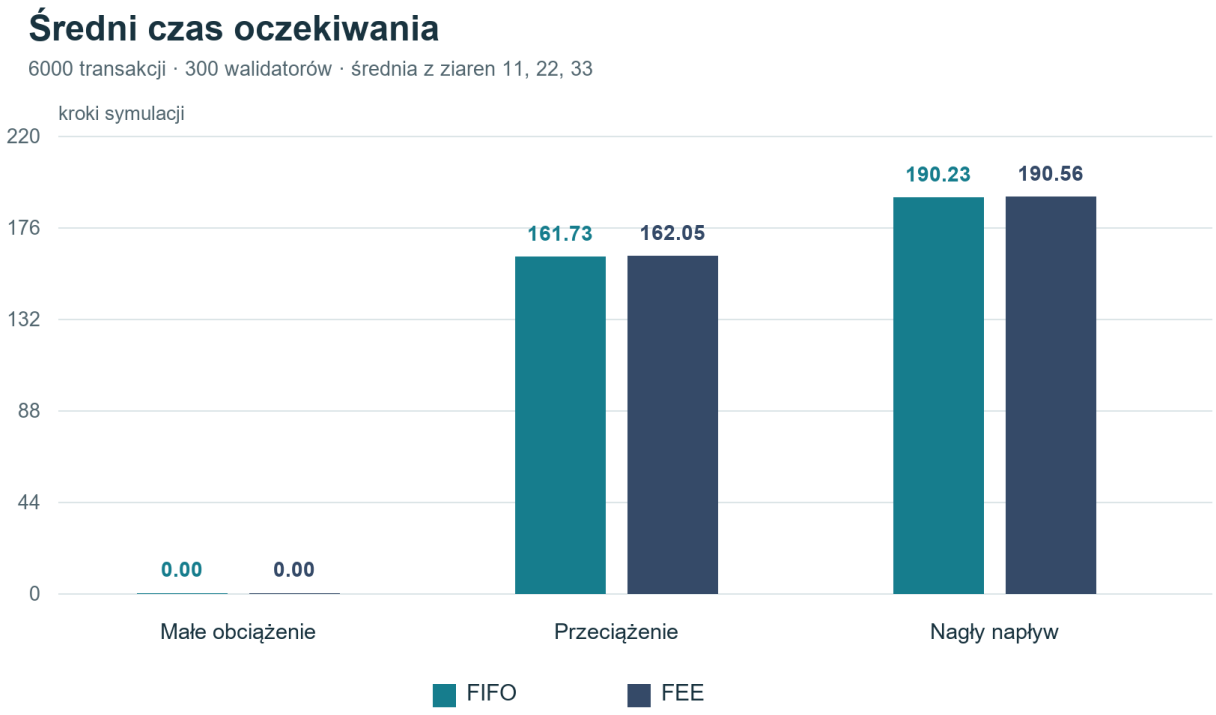
Scenariusz	Strategia	Seed	Śr. czekanie	Max	Terminowe	Spóźnione	Wykorzystanie %
Małe obciążenie	FIFO	11	0.000	0	6000	0	39.06
Małe obciążenie	FEE	11	0.000	0	6000	0	39.06
Małe obciążenie	FIFO	22	0.000	0	6000	0	39.06
Małe obciążenie	FEE	22	0.000	0	6000	0	39.06
Małe obciążenie	FIFO	33	0.000	0	6000	0	39.06
Małe obciążenie	FEE	33	0.000	0	6000	0	39.06
Przeciążenie	FIFO	11	161.728	333	787	5213	94.98
Przeciążenie	FEE	11	162.054	384	1014	4986	94.98
Przeciążenie	FIFO	22	161.728	333	787	5213	94.98
Przeciążenie	FEE	22	162.054	384	1014	4986	94.98
Przeciążenie	FIFO	33	161.728	333	787	5213	94.98
Przeciążenie	FEE	33	162.054	384	1014	4986	94.98
Nagły napływ	FIFO	11	190.227	391	690	5310	95.20
Nagły napływ	FEE	11	190.564	391	361	5639	95.43
Nagły napływ	FIFO	22	190.227	391	690	5310	95.20
Nagły napływ	FEE	22	190.564	391	361	5639	95.43
Nagły napływ	FIFO	33	190.227	391	690	5310	95.20
Nagły napływ	FEE	33	190.564	391	361	5639	95.43

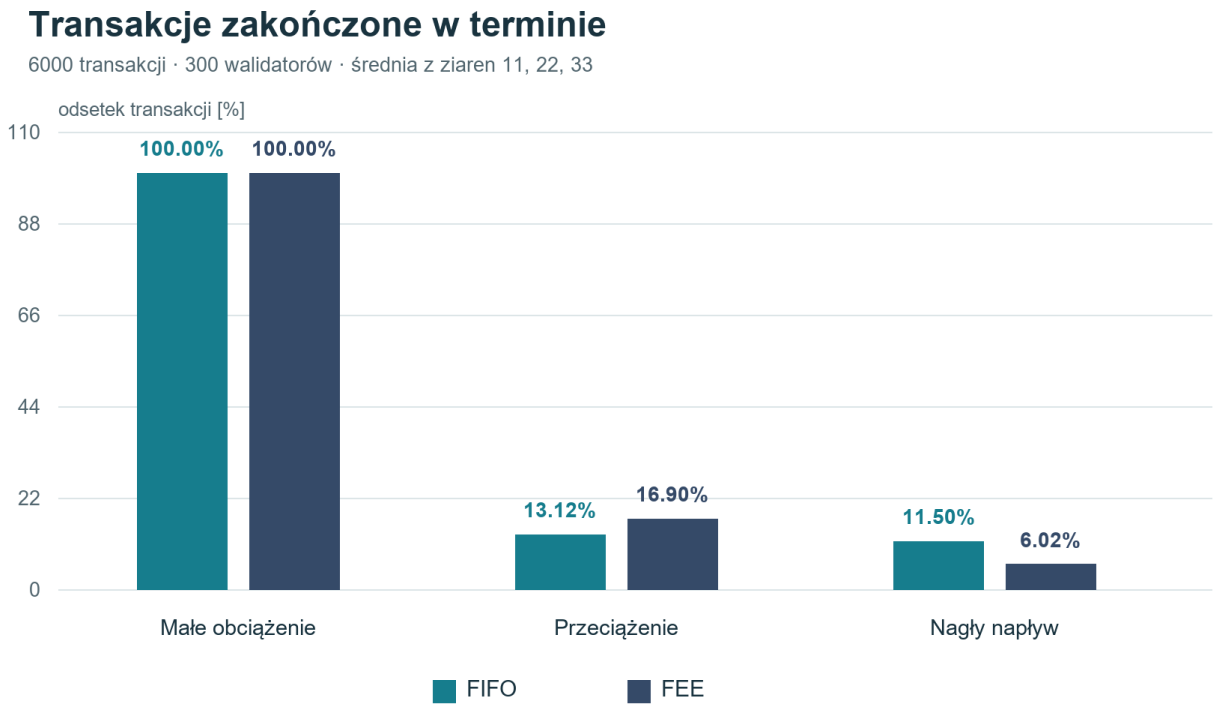
Wpływ ziarna

Wykonawcy mają jednakową szybkość. Zmiana ID wolnego wykonawcy nie zmienia czasu wykonania transakcji. Dlatego dla ustalonej strategii zmiana ziarna zachowuje harmonogram i zbiorcze miary oczekiwania oraz terminowości. Zmienia przydziały ID i rozkład wykorzystania pojedynczych walidatorów.

Przy małym obciążeniu w FIFO największe wykorzystanie jednego wykonawcy wyniosło 69,16%, 70,51% i 66,73% dla kolejnych ziaren. Średnia całej grupy pozostała 39,06%. Przydziały odtwarzamy w tym samym środowisku programu; uniform_real_distribution może dawać inne przekształcenie losowań w innej bibliotece standardowej.

6. Wykresy





Wykresy pokazują średnie z trzech ziaren. Dla przedstawionych miar wartości każdego ziarna są identyczne. Są to pomiary przebiegu w krokach, a nie pomiary czasu działania komputera w sekundach.

7. Porównanie i wnioski

Przy małym obciążeniu obie strategie mają zerowe oczekiwanie i 100% terminowości. Średnie wykorzystanie wynosi 39,06%. Gdy wolni wykonawcy przyjmują zadania od razu, zmiana priorytetu nie skraca oczekiwania.

W przeciążeniu FIFO ma średnie oczekiwanie 161,73 kroku, a FEE 162,05. FEE zwiększa terminowość z 787/6000, czyli 13,12%, do 1014/6000, czyli 16,90%. Jednocześnie podnosi maksimum oczekiwania z 333 do 384.

Przy nagłym napływie FIFO kończy terminowo 690 transakcji, czyli 11,50%, a FEE 361, czyli 6,02%. Wysokie opłaty dostają pierwszeństwo, ale mają w danych krótszy zapas deadline. FEE nie wybiera najbliższego terminu i może obniżyć łączną terminowość.

Scenariusz	Strategia	Grupa fee	Średnie oczekiwanie	Terminowe %
Przeciążenie	FIFO	1 do 25	162,51	18,55
Przeciążenie	FIFO	75 do 100	161,68	7,38
Przeciążenie	FEE	1 do 25	300,30	5,32
Przeciążenie	FEE	75 do 100	28,00	50,45
Nagły napływ	FIFO	1 do 25	191,14	15,82
Nagły napływ	FIFO	75 do 100	190,16	6,81
Nagły napływ	FEE	1 do 25	344,93	0,00
Nagły napływ	FEE	75 do 100	42,47	22,96

Grupa fee 1 do 25 liczy 1391 transakcji, a grupa 75 do 100 liczy 1572. Tabela pokazuje ziarno 11; pozostałe ziarna dają te same miary grup. Transakcje ze środkowego zakresu fee nie są w tej tabeli, lecz pozostają w pomiarach całego przebiegu.

FEE spełnia cel pierwszeństwa wysokich opłat. Ceną jest dłuższe oczekiwanie niskich opłat. FIFO zachowuje kolejność nadejścia. Z naszych badań nie wynika, że jedna strategia zawsze zapewnia mniejsze oczekiwanie i więcej dotrzymanych terminów. Ocena zależy od wybranej miary i danych wejściowych.

8. Złożoność i ograniczenia

N oznacza liczbę transakcji, K liczbę walidatorów, a T liczbę zwiększeń czasu. B to odstęp raportów, domyślnie 100. Liczba raportów R nie przekracza floor(N/B)+1.

Operacja	Koszt czasu
FIFO push, pop, front	O(1).
FEE push i pop; top	O(log(N+1)); top O(1).
Sprawdzenie ID i sortowanie wejścia	O(N log(N+1) + K log(K+1)), przy stałej liczbie pól.
Skany wykonawców w czasie	O((T+1)K), także dla pustego przebiegu.
Wybór wykonawcy dla N przydziałów	O(NK).
Końcowe wykorzystanie	O(NK): K wykonawców i N wyników.
Raporty narastające	O(RK).
Całość silnika	O(N log(N+1) + K log(K+1) + (T+1)K + NK + RK).
Pamięć silnika	O(N+K).

Ponieważ R jest O(N+1), raporty nie zmieniają ogólnej granicy. Dwa przebiegi mnożą koszt przez stałą. GUI przechowuje indeksy wierszy O(N). Sortowanie widoku ma O(N log(N+1)) porównań przy dostępnym dodatkowym buforze; bez niego stable_sort może wymagać O(N log²(N+1)). Skany wykonawców są w Symulator.cpp, a końcowe statystyki w Statystyki.cpp.

Długie przerwy w nadejściach zwiększają T, bo pętla sprawdza każdy krok. Przy ciągłym napływie wysokich opłat FEE może głodzić niskie opłaty. Nasze pliki są skończone, więc wszystkie zadania kończymy; mierzymy długie oczekiwanie.

Stały stake zmienia podział pracy między ID, ale nie zwiększa szybkości wykonania. Nie badamy nagród, kar, awarii, różnej wydajności stanowisk ani komunikacji sieciowej. Dane syntetyczne i trzy ziarna nie opisują wszystkich warunków pracy.

9. Sprawdzenie poprawności

Mały przykład daje FIFO 1,2,3 i FEE 2,3,1, średnie oczekiwanie 2, maksimum 4 i wykorzystanie 100%. Testy obejmują remisy, granice losowania stake, pomijanie zajętych, puste i błędne wejście, dokładny deadline, przerwy w pracy i raporty 100/200/205. Sprawdzamy także równoczesne zakończenia i pracę w toku.

Dla wszystkich 18 przebiegów testy ponownie liczą miary ze 108000 zapisanych rekordów. Sprawdzają unikalność ID, czasy, brak nakładania pracy i wykorzystanie. Ponowne wygenerowanie danych i wyników daje te same sumy SHA-256. Okno testujemy jako rzeczywisty program Windows, łącznie z błędem zapisu, zmianą wejścia poza programem, sortowaniem i zamknięciem w trakcie pracy.

10. Uruchomienie i odtworzenie

Na Windows uruchamiamy symulator_gui.exe albo symulator.exe z folderu projektu. GUI pokazuje wybór plików, zakładki FIFO/FEE i dziewięć pól wyniku. Obliczenia działają w osobnym wątku. Kliknięcie nagłówka sortuje widok liczbowo, a przycisk przywraca kolejność zakończenia.

Konsola domyślnie czyta konfiguracja.txt i zgloszenia.txt. Parametry --konfiguracja, --zgloszenia i --wyniki pozwalają wskazać ścieżki. --ziarno zmienia ziarno wyboru wykonawców, --co odstęp raportów, --cicho wyłącza komunikaty, a --pomoc podaje składnię. GUI zapisuje do wyniki obok wybranego pliku transakcji; konsola do bieżącego folderu lub wskazanego przez --wyniki. Kolejne uruchomienie nadpisuje dwa pliki w tym folderze.

Kod budujemy poleceniem .\build.cmd. Same testy uruchamiamy przez .\testy.cmd. Potrzebny jest GCC/MinGW z C++17 i windres dostępne w PATH. Opcja .\build.cmd --tests kompiluje programy i uruchamia testy. GUI używa standardowych kontrolek Win32 i jest przeznaczone dla Windows. Do uruchomienia gotowych exe nie potrzeba Pythona ani Qt.

Eksperymenty.exe odtwarza i nadpisuje konfigurację, domyślne zgłoszenia, trzy scenariusze, CSV i wyniki badań. Ręczny mały przykład pozostaje w przyklady/. W paczce testy są wyłącznie w C++. Sprawdzają losowanie, raporty, sortowanie, pliki, statystyki i 18 przebiegów. Skrypty robocze oraz źródła opisów nie są potrzebne do uruchomienia programu ani testów.

11. Materiały i wymagania przedmiotu

Kod i model dotyczą K_W05, porównanie badań K_U05, decyzje o algorytmach K_U08, analiza O() K_U09, a działający program K_U15. Sprawozdanie i przygotowanie pokazu wspierają K_K09 oraz K_K10. Wyjaśnienia autorów podczas obrony ocenia prowadzący.

Do pokazu przygotowaliśmy mały przykład, duży zestaw, plan do pięciu minut i materiały do nauki. Próba zespołu ma potwierdzić czas oraz zrozumienie kodu. Pliki do oddania są oddzielone od pomocniczych pytań i ćwiczeń.

Materiały źródłowe projektu

Kod C++, pliki wejściowe, porownanie.csv, wykorzystanie.csv, grupy_fee.csv, pełne wyniki eksperymentów oraz analiza złożoności z rozdziału 8. Diagramy pochodzą z naszej tablicy FigJam: https://www.figma.com/board/ITzrf0DeaTyMBG1xbEF7J7/Untitled?node-id=0-1 . Kopie do pracy bez internetu są w diagramy.pdf.